

Part 2: Embedded System Development**Lab 5: Keypad Integration with I2C LED/OLED Display****Goals**

1. Get familiar with the CubeMX code generator and Keil uVision software development environment
2. Create a multi-stage C project for STM32F4 discovery kit and program the kit
3. Learn basics of GPIO input and output configuration: input/output
4. Program GPIO registers to perform simple digital I/O input and output

Grading Rubrics (Total = 20 points, with 2 bonus points)

1. Documentation and Maintainability (5 points)
2. Functionality and Correctness (5 points)
3. Lab Demonstration (10 points)
4. Extra features (2 points)

Lab preparation recommendation:

1. Read Lecture note uploaded to D2L
2. Read Textbook **Chapter 14 GPIO**.
3. Watch Youtube Tutorials (<http://web.eece.maine.edu/~zhu/book/tutorials.php>)
4. Read the datasheet 0.96 I2C based OLED (**SSD1306 controller**) Display.

Lab assignment

- Following Book Chapter 14 and implementing a C program that displays 0-D including the the star (*) and hash (#) symbol.
- Implement displaying a moving “Hello World” that will start moving left to right with push of A button and stop with B button
- Do something cool. The following gives a few examples, but you are not limited to this. ***Creative ideas are always encouraged.***
 - Any cool implementation with your same circuit or different circuit.

GPIO data registers:

- 16-bit memory-mapped data registers for each port GPIOx x = A...I (GPIOA, GPIOB, ..., GPIOI)
- GPIOx_IDR :
 - Data input through the 16 pins
 - Read-only
- GPIOx_ODR
 - Write data to be output to the 16 pins
 - Read last value written to ODR
 - Read/write (for read-modify-write operations)
- C examples:
 - GPIOA->ODR = 0x007F; //send data to output pins
 - N = GPIOA->IDR; //copy data from in pins to N

Code Example

The code provided below is an STM32 microcontroller application at top level using the HAL (Hardware Abstraction Layer) library. This code initializes and runs an embedded system where a key pressed on a 4x4 keypad is detected and displayed on a 0.96" OLED screen using the I2C protocol.

```
int main(void) {
    HAL_Init();
    SystemClock_Config(); // Configure the system clock
    MX_GPIO_Init();       // Initialize GPIO (for keypad)
    MX_I2C1_Init();        // Initialize I2C for OLED

    OLED_Init();           // Initialize OLED display
    Keypad_Init();         // Initialize keypad

    char key = '\0';       // Variable to store the pressed key

    while (1) {
        key = Keypad_Read(); // Read the keypad

        if (key != '\0') { // If a key is pressed
            OLED_Display_Char(key); // Display it on the OLED
        }

        HAL_Delay(100); // Small delay to avoid key bouncing issues
    }
}
```

Code Comments and Documentation

Program comments are used to improve code readability, and to assist in debugging and maintenance. A general principal is ***“Structure and document your program the way you wish other programmers would”*** (McCann, 1997).

The book titled *“The Elements of Programming Style”* by Brian Kernighan and P. J. Plauger gives good advices for beginners.

1. **Format your code well.** Make sure it's easy to read and understand. Comment where needed but don't comment obvious things it makes the code harder to read. If editing someone else's code, format consistently with the original author.

2. Every program you write that you intend to keep around for more than a couple of hours ought to have documentation in it. Don't talk yourself into putting off the documentation. A program that is perfectly clear today is clear only because you just wrote it. Put it away for a few months, and it will most likely take you a while to figure out what it does and how it does it. If it takes you a while to figure it out, how long would it take someone else to figure it out?
3. **Write Clearly** - don't be too clever - don't sacrifice clarity for efficiency.
4. **Don't over comment.** Use comments only when necessary.
5. Format a program to help the reader understand it. **Always Beautify Code.**
6. Say what you mean, **simply and directly.**
7. **Don't patch bad code** - rewrite it.
8. **Make sure comments and code agree.**
9. Don't just echo code in comments - **make every comment meaningful.**
10. **Don't comment bad code** - rewrite it.
11. **The single most important factor in style is consistency.** The eye is drawn to something that "doesn't fit," and these should be reserved for things that are actually different.

References:

1. Pinout: <https://stm32-base.org/boards/STM32F103C8T6-Blue-Pill.html>
2. STLink V2 Debugger: <https://stm32-base.org/boards/Debugger-STM32F103C8U6-STLINKV2>
3. STM32 backgrounds: <https://stm32-base.org/>
4. STM32 Tutorials:
<https://www.youtube.com/watch?v=yvcAiSN7u8I&list=PLtuqBdbSL-DvXMyzhGnODxjYmrH9v-PPY>
5. Sample STM32 labs: <https://web.eece.maine.edu/~zhu/book/lab.php>
6. Keil Installation Tutorial:
https://web.eece.maine.edu/~zhu/book/lab_tutorial/Tutorial_1_Keil_Installation.pdf

CSE331 Lab 5
Lab Demo

Student Names: _____

Student IDs: _____

Time & Date: _____

1. Submit your code
 - Submit and maintain your code in printed form (Only the main.c file)
 - Make sure to comment your codes appropriately (see the requirements of code comments and documentation discussed previously in this lab handout.)
2. Demo your implementation to your lab Instructor
3. Show Lab Instructor that you can make working a keypad reading embedded system.
4. Write your answer to the following questions (Lab Instructor may grade them during your lab session).
 - What modification do you require to make a start and stop your moving “Hello World” display.
 - How many segment displays will be required for “Hello World”.
 - Write a simple control flow diagram for your moving display.